

Comparison of Linear and Nonlinear Dimensionality Reduction Methods for Image Reconstruction – Final Update

Presented By: Taylor Davis and Kaleb Coleman

Problem Reminder



HIGH-DIMENSIONAL IMAGES
ARE EXPENSIVE TO STORE
AND PROCESS



DIMENSIONALITY
REDUCTION COMPRESSES
IMAGES INTO SMALLER
REPRESENTATIONS



PCA IS CLASSICAL & LINEAR



AUTOENCODERS ARE
NEURAL & NONLINEAR



QUESTION: WHICH
RECONSTRUCTS IMAGES
BETTER UNDER EQUAL
COMPRESSION?



Methods Overview

PCA

- Linear Projection
- No Gradient Training
- Parameter Efficient

Autoencoder

- Nonlinear Encoder/Decoder
- Learns Latent Representation
- More flexible, but more expensive

Diffusion

Bonus

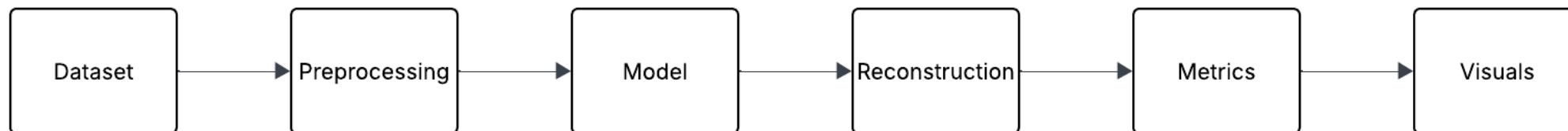


Implementation Update

- PCA Module
- AE Module
- Full Comparison Module
- Diffusion Bonus Module

```
✓ FINAL_PROJECT
> .cache
> .venv
> autoencoders
> comparison
> data
> diffusion
> pca
> slurm
> .gitignore
M↓ README.md
P!P requirements.txt
Python train.py
```

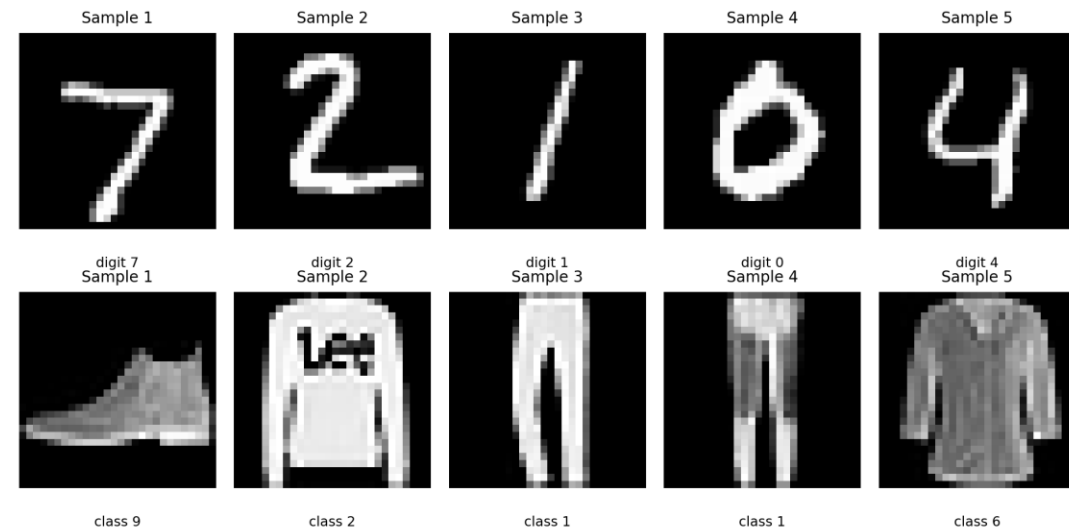
Project Flow



Data and Preprocessing

- Grayscale 28x28
- Tensor Conversion
- Normalized to 0-1
- Training and Testing Split for PCA vs. AE
- Shape: $N \times 1 \times 28 \times 28$

Datasets After Preprocessing



PCA Implementation

```
for n_components in args.components:
    pca = PCA(
        n_components=n_components,
        svd_solver="randomized",
        random_state=args.random_seed,
    )
    transformed = pca.fit_transform(X_train)

    reconstructed = pca.inverse_transform(pca.transform(X_test))
    reconstructed = np.clip(reconstructed, 0.0, PIXEL_MAX)
```

Core Feature	Meaning
<code>pca.fit(X_train)</code>	Make PCA model with a set number of latent dimensions, then fit to training set
<code>latent = pca.transform(X_test)</code>	Project test image into lower dimension
<code>reconstructed = pca.inverse_transform(latent)</code>	Recreate the image

AE Implementation

```
class FullyConnectedAutoencoder(nn.Module):
    """Fully connected autoencoder for 28x28 grayscale images."""

    def __init__(self, latent_dim: int) -> None:
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28 * 28, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim),
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, 28 * 28),
            nn.Sigmoid(),
        )

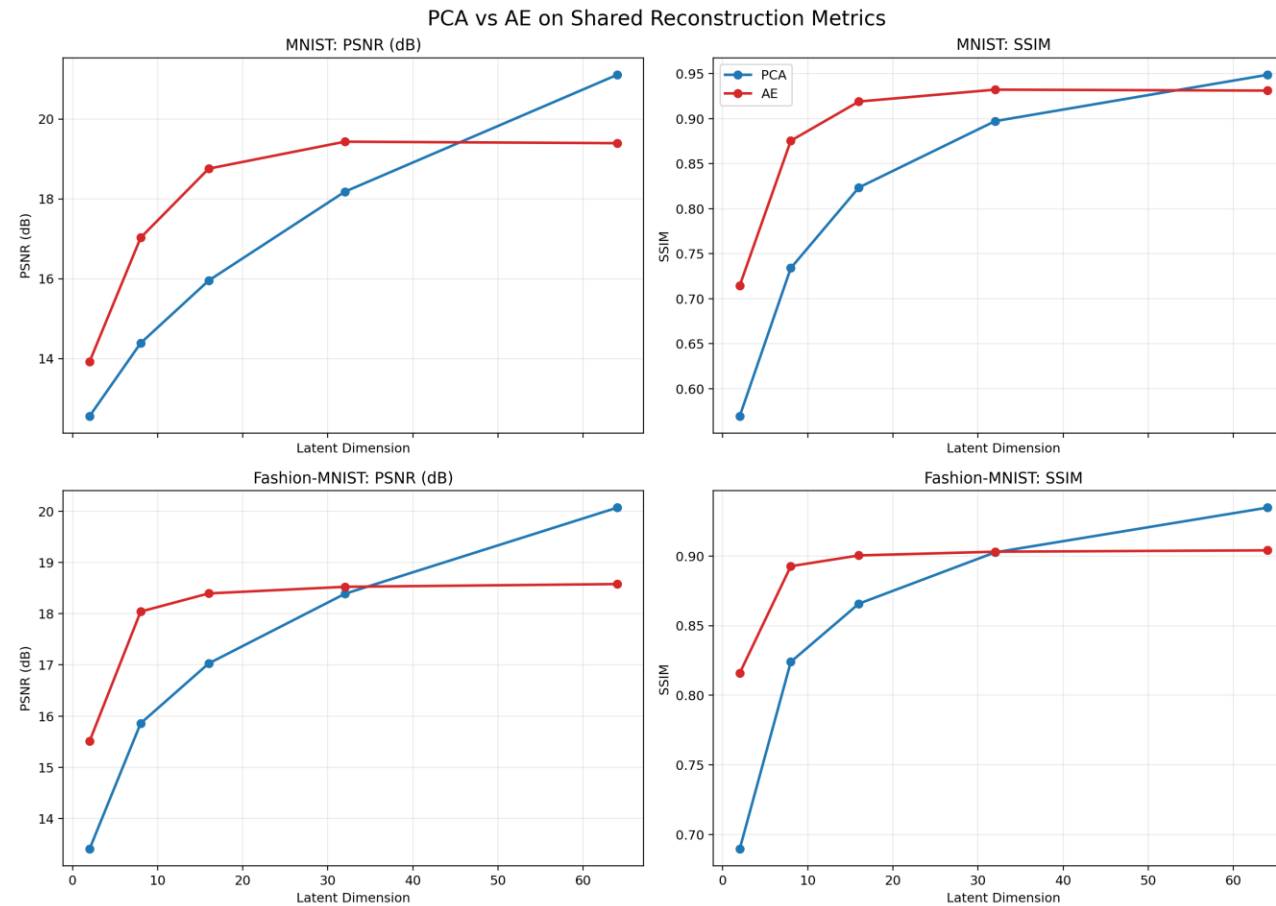
    def encode(self, x: torch.Tensor) -> torch.Tensor:
        return self.encoder(x)

    def decode(self, latent: torch.Tensor) -> torch.Tensor:
        decoded = self.decoder(latent)
        return decoded.view(-1, 1, 28, 28)

    def forward(self, inputs: torch.Tensor) -> torch.Tensor:
        latent = self.encode(inputs)
        return self.decode(latent)
```

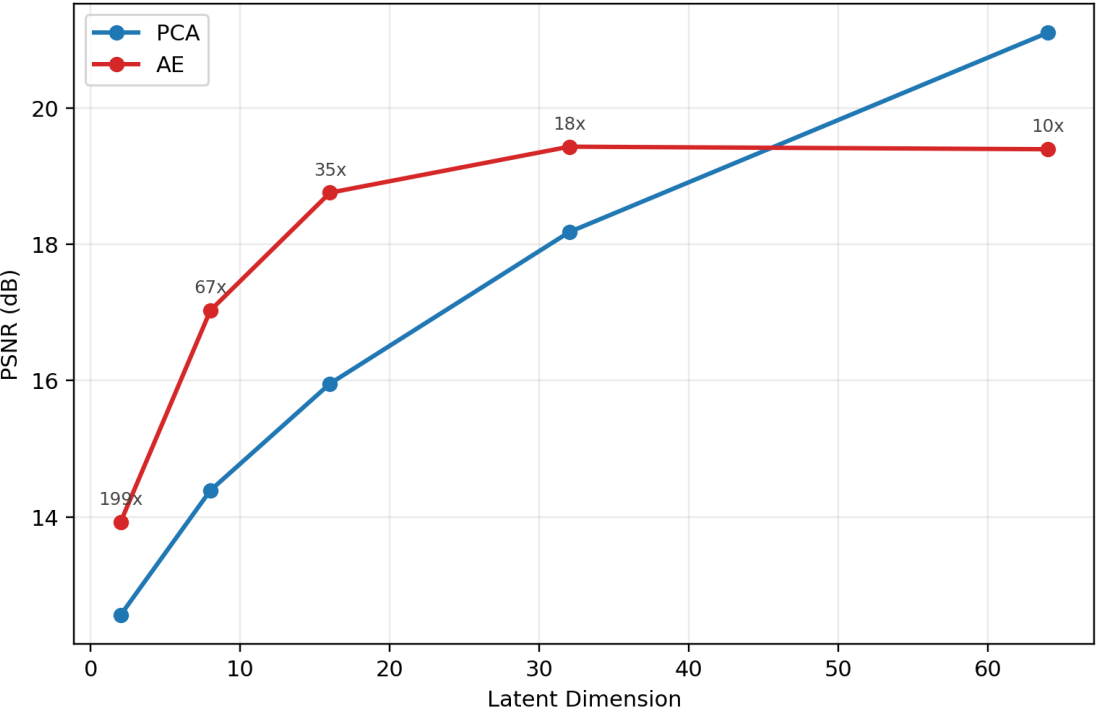
Core Feature	Meaning
self.encoder = nn.Sequential...	Compresses each 28x28 image into a smaller latent representation that keeps the most useful features
self.decoder = nn.Sequential...	Reconstructs the image from the latent code, expanding it back into pixel space
forward()	Defines the full reconstruction path

Initial Experimental Results: Quantitative

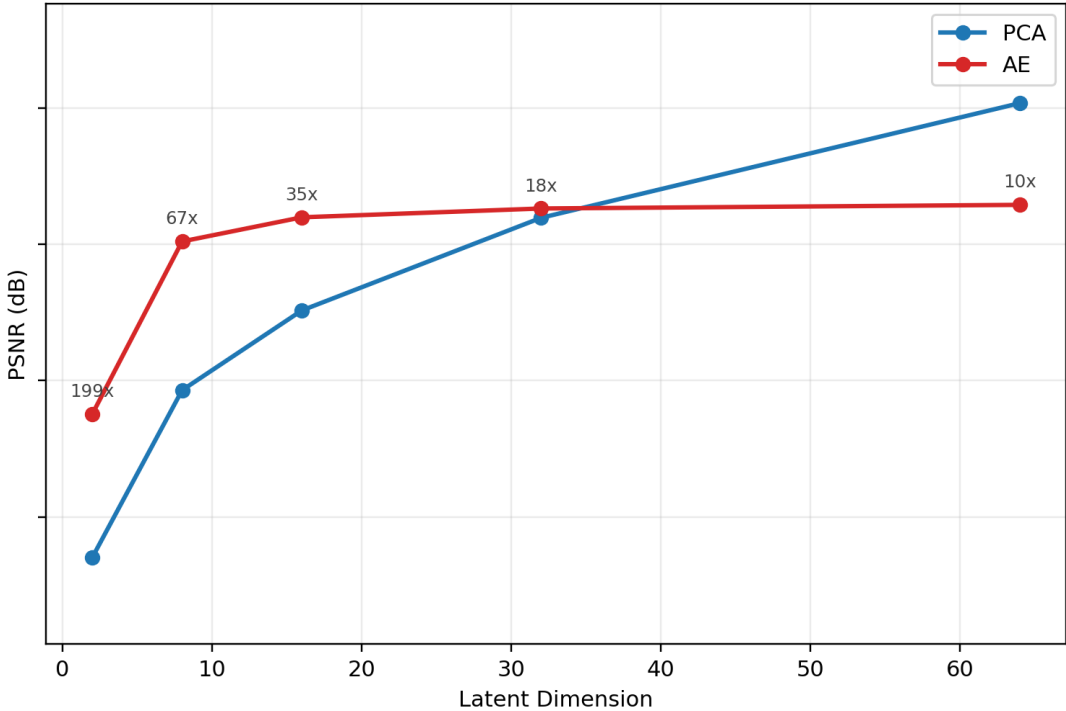


Tradeoffs

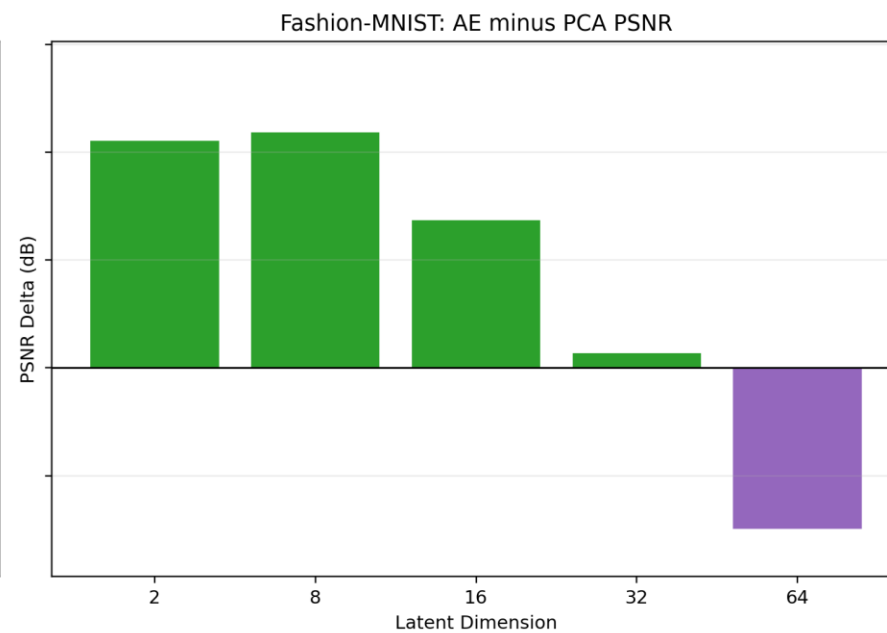
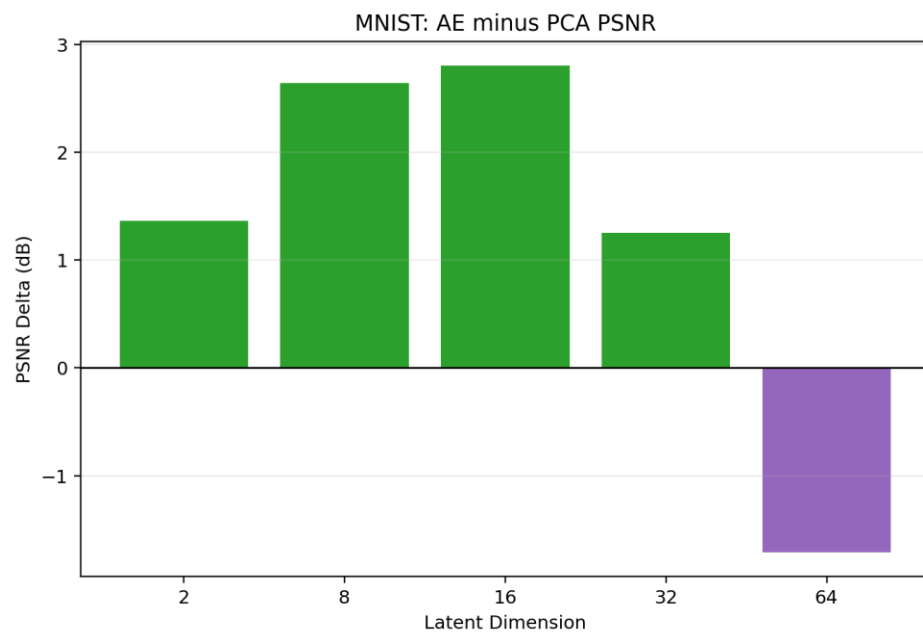
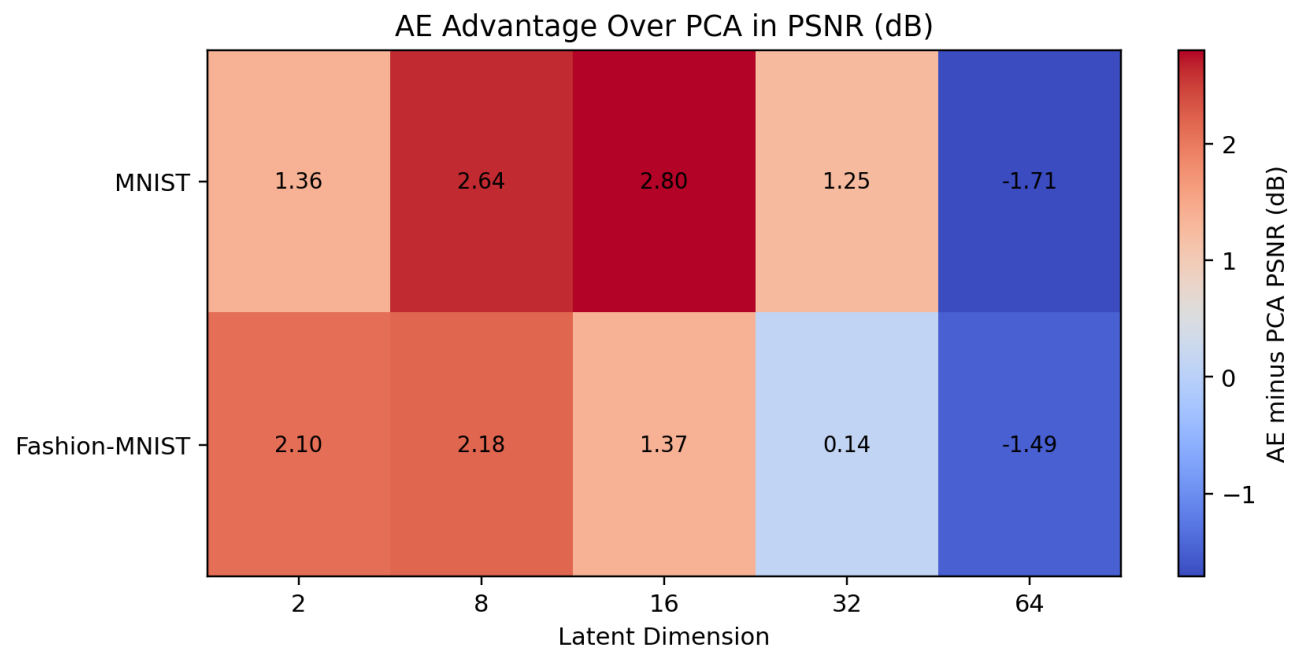
MNIST: Quality vs Model Cost



Fashion-MNIST: Quality vs Model Cost

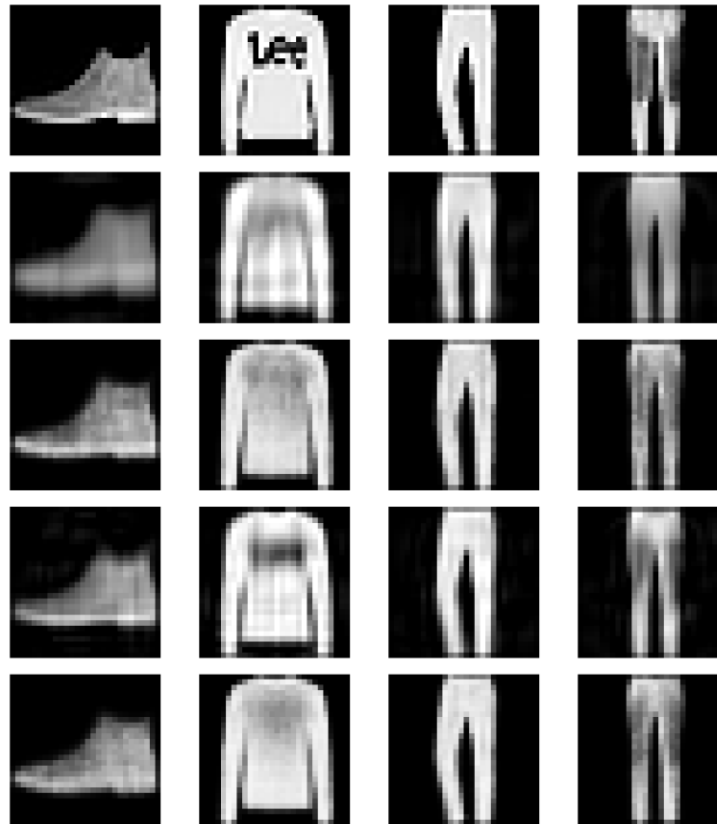


Details



Initial Experimental Results: Visuals

Fashion-MNIST: Side-by-Side Reconstruction Quality

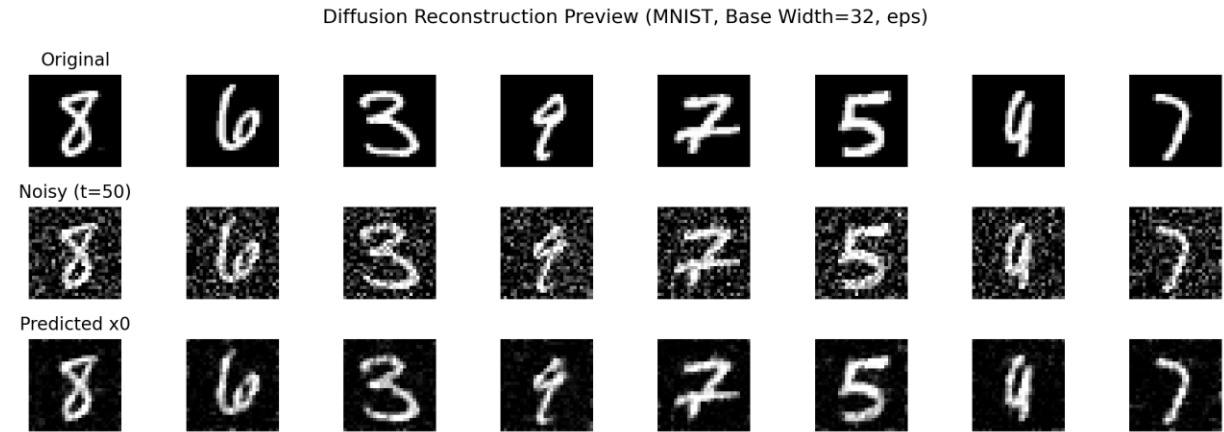
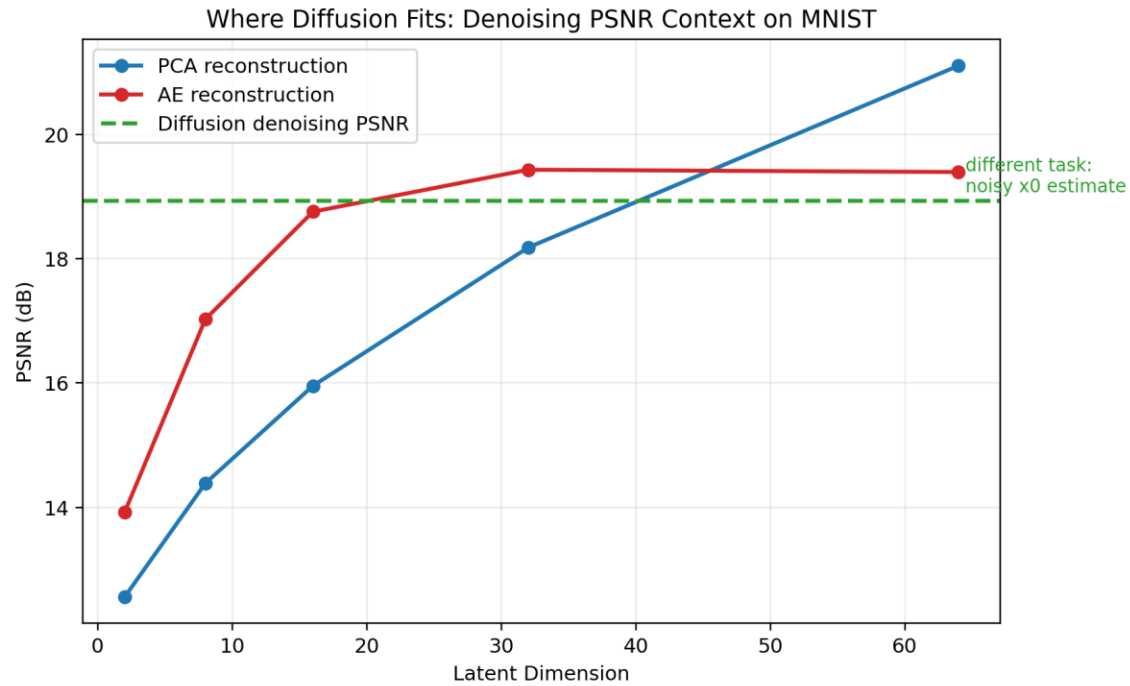




Interpretations and Tradeoffs

- ✓ AE beats PCA at small and medium latent sizes on both MNIST and Fashion-MNIST
 - ✓ PCA wins again at latent size 64
 - ✓ PCA is much more parameter-efficient and easier to train
 - ✓ Best choice depends on the goal:
 - ✓ Use AE when the bottleneck is tight and reconstruction quality matters most
 - ✓ Use PCA when simplicity, speed, and efficiency matter
- 

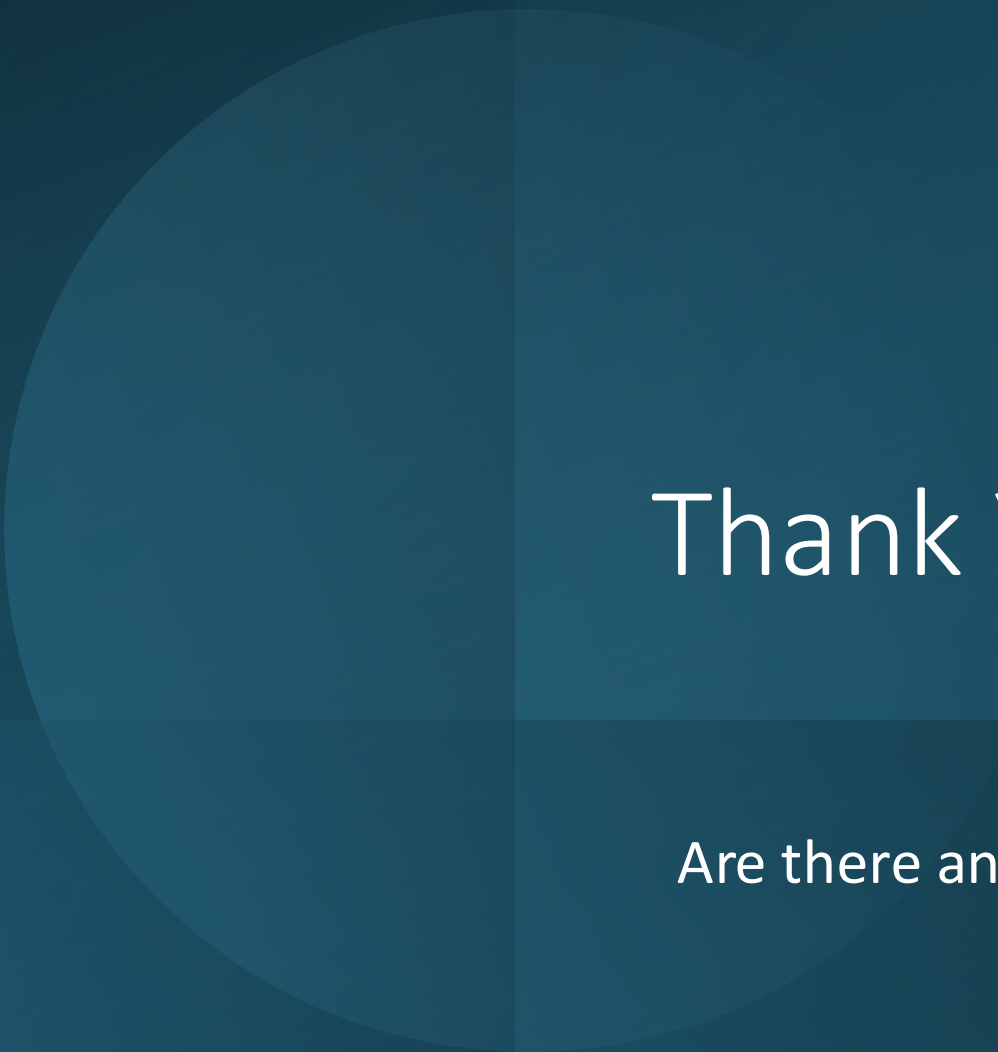
Bonus: Diffusion



Bonus: Diffusion



Limitations/State of Project/Website/GitHub



Thank You

Are there any questions?